

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №1
по дисциплине «Методы тестирования программного
обеспечения»
«Инспекция кода»

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Курочкин М. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Изучение методов ручного тестирования	5
2.1 Инспекция кода	5
2.2 Сквозные просмотры	6
2.3 Проверка за столом	6
2.4 Рецензирование	7
3 Описание тестируемой программы	8
3.1 Спецификация	8
3.2 Исходный код программы	9
4 Описание проведенной инспекции кода	13
4.1 Протокол заседания	13
4.2 Полученные рекомендации	16
4.3 Исправленный код программы	17
5 Заключение	21
Список литературы	22

Введение

В данной лабораторной работе представлено описание использования одного из методов ручного тестирования – инспекция кода.

Тестирование ПО – это процесс проверки соответствия фактического поведения программы ожидаемому, с целью выявления дефектов, подтверждения корректности функционала.

Ручное тестирование ПО – это вид тестирования программного обеспечения, при котором тестировщик выполняет тестовые сценарии и проверяет функциональность приложения вручную, без использования средств автоматизации.

1 Постановка задачи

При выполнении данной лабораторной работы требуется провести инспекцию кода, выступая в роли разработчика программы. Задачи:

1. изучить методы ручного тестирования;
2. провести инспекцию кода;
3. проанализировать результаты тестирования;
4. исправить программу в соответствии с рекомендациями специалиста по тестированию.

2 Изучение методов ручного тестирования

Основными методами ручного тестирования являются инспекция кода, сквозной просмотр и тестирование удобства использования. Эти методы могут применяться как ко всему приложению, так и к отдельным его блокам.

2.1 Инспекция кода

Инспекция кода – это набор процедур и методик обнаружения ошибок путем анализа (чтения) кода группой специалистов. Группа специалистов вручную анализирует исходный код программы без её запуска с целью выявления дефектов и логических ошибок.

Обычно в состав группы инспектирования кода входят 4 человека: координатор, программист, проектировщик программы, специалист по тестированию. Среди этих участников не должно быть программиста, который является автором тестируемой программы.

Координатор должен быть квалифицированным программистом, но не автором тестируемой программы, детальное знание которой от него не требуется. В его обязанности входит следующее: раздача материалов участникам заседания инспекционной группы и составление плана его проведения; проведение заседания; запись всех обнаруженных ошибок; последующая проверка того, что обнаруженные ошибки устранены.

Вторым участником группы является программист, а остальными – проектировщик программы (если это не сам программист) и специалист по тестированию. Последний должен не только хорошо ориентироваться в методах тестирования ПО, но и знать наиболее распространенные типы ошибок.

Порядок проведения заседания по инспекции кода

За несколько дней до заседания координатор раздает листинг программы и спецификацию проекта всем участникам группы. Это делается для того, чтобы к моменту проведения заседания участники успели ознакомиться с необходимыми материалами.

Во время заседания программист рассказывает присутствующим о логике работы программы, подробно останавливаясь на каждой инструкции. В это время другие участники заседания должны задавать ему вопросы, которые поспособствуют выявлению возможных ошибок.

Участники заседания анализируют код программы, сверяясь с составленными на основе имеющегося опыта контрольными списками наиболее распространенных ошибок.

Координатор обязан направлять дискуссию в конструктивное русло и следить за тем, чтобы участники концентрировали свое внимание не на устранении ошибок, а на их обнаружении.

По окончании заседания программисту передается список найденных ошибок. Если этот список достаточно длинный или устранение ошибок требует

внесения существенных изменений в программу, координатор может принять решение о повторной инспекции кода после того, как ошибки будут исправлены. Список анализируется и после разбиения ошибок на категории дополняет имеющийся контрольный список, что позволяет повысить эффективность будущих инспекций кода.

2.2 Сквозные просмотры

Сквозной просмотр кода представляет собой совокупность процедур и методов обнаружения ошибок в коде путем его просмотра участниками группы тестирования. При сквозном просмотре используются немного отличающиеся от инспекции кода методы.

Сквозной просмотр, как и инспекция кода, проводится в форме заседания. Число участников группы составляет 3-5 человек. Среди участников: координатор, секретарь, который записывает все найденные ошибки, тестировщик, программист.

За несколько дней до заседания точно так же, как при инспекции кода, участникам раздаются материалы, чтобы они могли заблаговременно с ними ознакомиться.

Само заседание проводится иначе, чем при инспекции кода. Тестировщик приходит на заседание с подготовленными заранее наборами входных данных и соответствующих им ожидаемых результатов работы программы. Во время заседания участники в уме прогоняют работы программы для каждого набора, а затем сверяют результат с ожидаемым. Далее на основе результатов тестов участники заседания задают вопросы программисту.

2.3 Проверка за столом

Проверка за столом – это метод ручного тестирования, который может представлять собой инспекцию кода или сквозной просмотр, но выполняемый одним человеком. Этот человек будет просматривать код программы, проверяя его на видимые ошибки, сверяться с контрольным списком ошибок, прогонять через логику программы тестовые данные.

Такая проверка неупорядочена и хаотична, что приводит к неэффективности. К тому же тестирование своих собственных программ нарушает принцип тестирования, согласно которому тестирование программистом программ, написанных им же, неэффективно. Даже если проверку за столом будет производить человек, не являющийся автором программы, она будет менее эффективна, чем инспекция кода или сквозные просмотры.

2.4 Рецензирование

Рецензирование – это процедура анонимной оценки общих характеристик качества, обслуживаемости, расширяемости, удобства использования и ясности ПО.

Цель метода – предоставить программисту возможность получить стороннюю оценку результатов своей работы.

Администратор, выбираемый из числа программистов, отбирает группу из 6-20 участников, которые и будут рецензировать код. Каждый участник предоставляет 2 своих программы для рецензирования. Программы выбираются следующим образом: одну он считает лучшей по качеству, другую – худшей.

Программы распределяют случайным образом между участниками. Каждому достается четыре программы: две из числа лучших и две из числа худших. Рецензент не знает, какая из программ какой является. Далее рецензент оставляет замечания и рекомендации к программам. После этого участникам передают рецензии на их программы и статистику с данными о рейтинге их программ среди всех, чтобы они ознакомились и оценили свой уровень квалификации.

3 Описание тестируемой программы

Программа должна реализовывать хэш-таблицу, поддерживающую операции добавления, удаления и поиска слова, очистки и вывода всего словаря, а также меню для взаимодействия пользователя с программой. При некорректном вводе должно выводиться сообщение об ошибке.

Входные данные:

- n – номер операции из меню;
- s – слово, над которым выполняется операция.

Ограничения:

- $1 \leq n \leq 6$;
- Слова, вводимые при работе с хэш-таблицей, должны содержать только буквы русского или английского алфавитов, ' и -.

Выходные данные: результат выполнения операции: подтверждение добавления или удаления слова, сообщение о наличии слова в словаре, содержимое хэш-таблицы, сообщение об очистке словаря, сообщение об ошибке.

3.1 Спецификация

Спецификация программы представлена в Табл. 1.

Таблица 1. Спецификация программы

№	Входные данные	Результат	Реакция программы
1	$n=2, s=\text{"кот"}$	Сообщение: The word is added successfully!	Слово успешно добавлено в хэш-таблицу. Программа ожидает следующего ввода.
2	$n=2, s=\text{"кот"}$	Сообщение: The word is already in the dictionary.	Программа сообщила, что слово уже существует в хэш-таблице и ожидает следующего ввода.
3	$n=2, s=\text{"cat"}$	Сообщение: The word is added successfully!	Слово успешно добавлено в хэш-таблицу. Программа ожидает следующего ввода.

№	Входные данные	Результат	Реакция программы
4	n=3, s="кот"	Сообщение: The word is removed succesfully!	Слово успешно удалено из хэш-таблицы. Программа ожидает следующего ввода.
5	n=4, s="cat"	Сообщение: The word is in the dictionary.	Программа нашла слово в хэш-таблице и ожидает следующего ввода.
6	n=6	Сообщение: Bucket 236: cat	Программа вывела содержимое хэш-таблицы и ожидает следующего ввода.
7	n=10	Сообщение: Error!	Программа сообщила о некорректном вводе и ожидает следующего ввода.

3.2 Исходный код программы

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 #include <locale>
6 #include <windows.h>
7 #include <regex>
8 #include <cctype>
9
10 using namespace std;
11
12 class HashTable
13 {
14 private:
15     static const int TABLE_SIZE = 1000; // Размер таблицы
16     vector<vector<string>> table; // Метод цепочек для разрешения коллизий
17
18     int hashFunction(const string& key) {
19         const int p = 37;
20         int hash = 0;
21         for (size_t i = 0; i < key.size(); ++i) {
22             hash = (hash * p + static_cast<unsigned char>(key[i])) %
23                 TABLE_SIZE;
24         }
25         return hash;
26     }
27
28     void insert(const string& word) {
29         if (word.empty()) return;
30         int hash = hashFunction(word);
31         auto& chain = table[hash];
32         if (std::find(chain.begin(), chain.end(), word) == chain.end()) {

```

```

33     }
34 }
35
36 bool find(const string& word) {
37     int hash = hashFunction(word);
38     const auto& chain = table[hash];
39     return std::find(chain.begin(), chain.end(), word) != chain.end();
40 }
41
42 void remove(const string& word) {
43     int hash = hashFunction(word);
44     auto& chain = table[hash];
45     auto it = std::find(chain.begin(), chain.end(), word);
46     if (it != chain.end()) {
47         chain.erase(it);
48     }
49 }
50
51 bool isEmpty() {
52     bool isEmpty = true;
53     for (const auto& chain : table) {
54         if (!chain.empty()) {
55             isEmpty = false;
56             break;
57         }
58     }
59     return isEmpty;
60 }
61
62 void clear() {
63     for (auto& chain : table) {
64         chain.clear();
65     }
66 }
67
68 void printAll() {
69     for (int i = 0; i < TABLE_SIZE; ++i) {
70         if (!table[i].empty()) {
71             cout << "Bucket " << i << ": ";
72             for (const auto& word : table[i]) {
73                 std::cout << word << " ";
74             }
75             cout << endl;
76         }
77     }
78 }
79
80 public:
81     HashTable() : table(TABLE_SIZE) {}
82
83     void menu();
84 };
85
86 void HashTable::menu()
87 {
88     cout << "\n===== \n
89         \n<1> quit\
90         \n<2> insert a word\
91         \n<3> remove a word\
92         \n<4> find a word\

```

```

93         \n<5> clear the dictionary\
94         \n<6> print the dictionary";
95
96     string choice;
97     regex power("[0-9]+$");
98     while (true) {
99         cout << "\nChoose an option (an integer from 1 to 6): ";
100        cin >> choice;
101        if (regex_match(choice, power) && stoi(choice) >= 1 && stoi(choice)
102            <= 6) {
103            break;
104        }
105        else {
106            cout << "\nError!\n\n";
107        }
108    }
109    cout << "=====\n\n";
110
111    string word;
112    regex word_regex("[A-Za-zЁёА-Яа-я3'-]+");
113    if (stoi(choice) == 2 || stoi(choice) == 3 || stoi(choice) == 4) {
114        while (true) {
115            cout << "Enter a word: ";
116            cin >> word;
117            if (regex_match(word, word_regex)) {
118                break;
119            }
120            else {
121                cout << "\nError!\n\n";
122            }
123        }
124    }
125
126    switch (stoi(choice)) {
127    case 1:
128        return;
129
130    case 2:
131        if (find(word)) {
132            cout << "The word is already in the dictionary." << endl;
133            break;
134        }
135        insert(word);
136        cout << "The word is added succesfully!" << endl;
137        break;
138
139    case 3:
140        if (!find(word)) {
141            cout << "The word is not in the dictionary." << endl;
142            break;
143        }
144        remove(word);
145        cout << "The word is removed succesfully!" << endl;
146        break;
147
148    case 4:
149        if (find(word)) cout << "The word is in the dictionary." << endl;
150        else cout << "The word is not in the dictionary." << endl;
151        break;

```

```

152
153     case 5:
154         if (isEmpty()) {
155             cout << "The dictionary is already empty." << endl;
156             break;
157         }
158         clear();
159         cout << "The dictionary is cleared." << endl;
160         break;
161
162     case 6:
163         printAll();
164         break;
165     }
166
167     menu();
168 }
169
170 int main() {
171     setlocale(LC_ALL, "Russian");
172     SetConsoleCP(1251);
173
174     HashTable hashTable;
175     hashTable.menu();
176
177     return 0;
178 }

```

4 Описание проведенной инспекции кода

В заседании участвовали:

- Программист (разработчик программы) – Лаишевкина Анна;
- Специалист по тестированию – Шихалев Алексей;
- Секретарь – Лаишевкина Анна.

В начале заседания разработчиком программы была описана логика программы и представлена ее спецификация. В процессе заседания были заданы вопросы с целью выявления ошибок. Секретарем был составлен протокол заседания, где были записаны вопросы и ответы на них, а также рекомендации по исправлению ошибок.

4.1 Протокол заседания

1. На каком языке написана программа?

Ответ: на языке C++.

2. Используются ли переменные с неинициализированными значениями?

Ответ: нет.

3. Лежат ли индексы вне заданных границ?

Ответ: нет.

4. Есть ли нецелые индексы?

Ответ: нет.

5. Есть ли «подвешенные» обращения?

Ответ: нет.

6. Корректны ли атрибуты всех псевдонимов?

Ответ: да.

7. Соответствуют ли атрибуты записи и структуры?

Ответ: да.

8. Корректны ли атрибуты базированной памяти?

Ответ: да.

9. Соответствуют ли друг другу определения структуры, данные ей в различных процедурах?

Ответ: да.

10. Существуют ли какие-нибудь другие ошибки в операциях с индексацией или при обращении к массивам по индексу?
Ответ: нет.
11. Производятся ли вычисления неарифметических переменных?
Ответ: нет.
12. Производятся ли вычисления с использованием данных разного вида?
Ответ: да, в хэш-функции происходит приведение `char` к `unsigned char` и `int`.
13. Существуют ли вычисления переменных разной длины?
Ответ: нет.
14. Не меньше ли длина результата, чем длина вычисляемого значения?
Ответ: нет.
15. Возможно ли переполнение или потеря промежуточного результата при вычислении?
Ответ: нет.
16. Есть ли деление на нуль?
Ответ: нет.
17. Не выходит ли значение переменной за пределы установленного диапазона?
Ответ: нет.
18. Понятен ли порядок следования операторов?
Ответ: да.
19. Правильно ли осуществляется деление целых чисел?
Ответ: да.
20. Все ли переменные описаны?
Ответ: да.
21. Правильно ли инициализированы массивы и строки?
Ответ: да, вектор `table` инициализируется в конструкторе.
22. Правильно ли определены размер, тип и класс памяти?
Ответ: да.
23. Согласуется ли инициализация с классом памяти?
Ответ: да.
24. Сравниваются ли величины несовместимых типов?
Ответ: нет.

25. Сравняются ли величины различных типов?
Ответ: нет.
26. Корректны ли отношения сравнения?
Ответ: да.
27. Корректны ли булевские выражения?
Ответ: да.
28. Понятен ли порядок следования операторов?
Ответ: да.
29. Может ли значение индекса в переключателе превысить число переходов?
Ответ: нет.
30. Будет ли завершен каждый цикл?
Ответ: да.
31. Будет ли завершена программа?
Ответ: да.
32. Существует ли какой-нибудь цикл, который не выполняется из-за входных условий?
Ответ: нет.
33. Корректны ли возможные погружения в цикл?
Ответ: да.
34. Есть ли ошибка отклонения числа итераций от нормы?
Ответ: нет.
35. Соответствует ли формат спецификации операторам ввода-вывода?
Ответ: да.
36. Обнаруживаются ли ошибки ввода-вывода?
Ответ: да.
37. Существуют ли какие-нибудь текстовые ошибки в выходной информации?
Ответ: нет.
38. Равно ли число входных параметров числу аргументов?
Ответ: да.
39. Соответствуют ли атрибуты параметров и аргументов?
Ответ: да.

40. Равно ли число аргументов, передаваемых вызываемым модулям, числу параметров?
Ответ: да.
41. Правильно ли заданы число, атрибуты и порядок следования аргументов для встроенных функций?
Ответ: да.
42. Не изменяет ли подпрограмма аргументы, являющиеся только входными?
Ответ: нет.
43. Передаются ли в качестве аргументов константы?
Ответ: да, строковые литералы.
44. Есть ли в таблице перекрестных ссылок какие-нибудь переменные, на которые нет ссылок?
Ответ: нет.
45. Список атрибутов такой, который и ожидался?
Ответ: да.
46. Есть ли какие-нибудь предупреждения или информационные сообщения?
Ответ: да, программа выводит сообщения об ошибках и результатах операций.
47. Осуществляется ли контроль правильности входных данных?
Ответ: да, с помощью регулярных выражений.
48. Нет ли пропущенных функций?
Ответ: нет.

4.2 Полученные рекомендации

В результате инспекции кода были получены следующие рекомендации:

1. Заменить рекурсивный вызов `menu()` на цикл.
2. Добавить `const` к методам, не изменяющим состояние.
3. Добавить нормализацию регистра при вводе слова.
4. Улучшить сообщения об ошибках.
5. Оптимизировать проверку существования слова.

4.3 Исправленный код программы

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 #include <clocale>
6 #include <windows.h>
7 #include <regex>
8 #include <cctype>
9
10 using namespace std;
11
12 class HashTable
13 {
14 private:
15     static const int TABLE_SIZE = 1000; // Размер таблицы
16     vector<vector<string>> table; // Метод цепочек для разрешения коллизий
17
18     int hashFunction(const string& key) const {
19         const int p = 37;
20         int hash = 0;
21         for (size_t i = 0; i < key.size(); ++i) {
22             hash = (hash * p + static_cast<unsigned char>(key[i])) %
23                 TABLE_SIZE;
24         }
25         return hash;
26     }
27
28     void toLower(string& str) {
29         transform(str.begin(), str.end(), str.begin(),
30             [](unsigned char c) { return tolower(c); });
31     }
32
33     bool insert(const string& word) {
34         if (word.empty()) return false;
35
36         int hash = hashFunction(word);
37         auto& chain = table[hash];
38
39         if (std::find(chain.begin(), chain.end(), word) == chain.end()) {
40             chain.push_back(word);
41             return true;
42         }
43         return false;
44     }
45
46     bool find(const string& word) const {
47         int hash = hashFunction(word);
48         const auto& chain = table[hash];
49         return std::find(chain.begin(), chain.end(), word) != chain.end();
50     }
51
52     bool remove(const string& word) {
53         int hash = hashFunction(word);
54         auto& chain = table[hash];
55         auto it = std::find(chain.begin(), chain.end(), word);
56
57         if (it != chain.end()) {
58             chain.erase(it);
59         }
60     }
61 }
```

```

58         return true;
59     }
60     return false;
61 }
62
63 bool isEmpty() const {
64     for (const auto& chain : table) {
65         if (!chain.empty()) {
66             return false;
67         }
68     }
69     return true;
70 }
71
72 void clear() {
73     for (auto& chain : table) {
74         chain.clear();
75     }
76 }
77
78 void printAll() const {
79     bool empty = true;
80     for (int i = 0; i < TABLE_SIZE; ++i) {
81         if (!table[i].empty()) {
82             cout << "Bucket " << i << ": ";
83             for (const auto& word : table[i]) {
84                 cout << word << " ";
85             }
86             cout << endl;
87             empty = false;
88         }
89     }
90     if (empty) {
91         cout << "Dictionary is empty." << endl;
92     }
93 }
94
95 public:
96     HashTable() : table(TABLE_SIZE) {}
97
98     void menu();
99 };
100
101 void HashTable::menu()
102 {
103     regex power_regex("[0-9]+$");
104     regex word_regex("[A-Яa-яЁёA-Za-z0-9'-]+");
105
106     while (true) {
107         cout << "\n===== \n";
108         cout << "<1> quit\n";
109         cout << "<2> insert a word\n";
110         cout << "<3> remove a word\n";
111         cout << "<4> find a word\n";
112         cout << "<5> clear the dictionary\n";
113         cout << "<6> print the dictionary\n";
114         cout << "===== \n";
115
116         string choice;
117         int option = 0;

```

```

118
119 while (true) {
120     cout << "\nChoose an option (an integer from 1 to 6): ";
121     cin >> choice;
122
123     if (!regex_match(choice, power_regex)) {
124         cout << "Error: Input must contain only digits.\n";
125         continue;
126     }
127
128     option = stoi(choice);
129
130     if (option >= 1 && option <= 6) {
131         break;
132     } else {
133         cout << "Error: Number must be between 1 and 6.\n";
134     }
135 }
136
137 string word;
138 if (option == 2 || option == 3 || option == 4) {
139     while (true) {
140         cout << "Enter a word: ";
141         cin >> word;
142
143         toLower(word);
144
145         if (regex_match(word, word_regex)) {
146             break;
147         } else {
148             cout << "Error: Word contains invalid characters. Use
149                 letters, numbers, hyphens or apostrophes.\n";
150         }
151     }
152
153     cout << "-----\n";
154
155     switch (option) {
156     case 1:
157         cout << "Exiting program..." << endl;
158         return;
159
160     case 2:
161         if (insert(word)) {
162             cout << "The word '" << word << "' is added successfully!"
163                 << endl;
164         } else {
165             cout << "The word '" << word << "' is already in the
166                 dictionary." << endl;
167         }
168         break;
169
170     case 3:
171         if (remove(word)) {
172             cout << "The word '" << word << "' is removed successfully!"
173                 << endl;
174         } else {
175             cout << "The word '" << word << "' is not in the dictionary.
176                 " << endl;

```

```

173     }
174     break;
175
176     case 4:
177         if (find(word)) {
178             cout << "The word '" << word << "' is in the dictionary." <<
179                 endl;
180         } else {
181             cout << "The word '" << word << "' is not in the dictionary."
182                 << endl;
183         }
184         break;
185
186     case 5:
187         if (isEmpty()) {
188             cout << "The dictionary is already empty." << endl;
189         } else {
190             clear();
191             cout << "The dictionary is cleared." << endl;
192         }
193         break;
194
195     case 6:
196         printAll();
197         break;
198     }
199 }
200
201 int main() {
202     setlocale(LC_ALL, "Russian");
203     SetConsoleCP(1251);
204     SetConsoleOutputCP(1251);
205
206     HashTable hashTable;
207     hashTable.menu();
208
209     return 0;
210 }

```

5 Заключение

В ходе данной лабораторной работы был изучен один из методов ручного тестирования – инспекция кода.

В результате проведения инспекции кода были обнаружены некоторые недочеты, а также даны рекомендации по улучшению кода. В код были внесены изменения, соответствующие рекомендациям. Был заменен рекурсивный вызов `menu()` на цикл `while`; методы, не изменяющие состояние, были сделаны `const`; была добавлена нормализация регистра при вводе слова; были уточнены сообщения об ошибках; была оптимизирована проверка существования слова.

По итогу выполненной работы я сделала вывод, что методы ручного тестирования кода могут быть эффективными для улучшения читаемости и чистоты кода, а также для поиска недочетов, которые не всегда видны самому разработчику программы.

Список литературы

1. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.